

IBFT algoritam

Dokumentacija uz predmetni projekat

Predmet: Primenjeni algoritmi u upravljačkim sistemima

Minja Mrđen
E2 15/2025

1. Opis problema i cilj rada

Tema rada je implementacija i simulacija Istanbul Byzantine Fault Tolerance (IBFT) konsenzus algoritma u programskom jeziku Go. Problem koji se rešava jeste postizanje dogovora između više čvorova u distribuiranom sistemu, čak i kada se deo čvorova ponaša neispravno ili zlonamerno. Takvi čvorovi nazivaju se vizantijski čvorovi, jer mogu ćutati, slati pogrešne poruke ili različitim učesnicima slati različite vrednosti.

Cilj implementacije nije produkciona blockchain mreža, već edukativna simulacija koja pokazuje glavne delove IBFT algoritma: normalan tok poruka, promenu runde kada lider ne uspe, izbor bezbedne vrednosti pri promeni runde, čuvanje dokaza odluke i ponašanje sistema u različitim vizantijskim scenarijima.

IBFT može da toleriše najviše f vizantijskih čvorova ako važi uslov $n \geq 3f + 1$, gde je n ukupan broj validatora. Za najčešće korišćeni slučaj u simulacijama, $n = 4$, sistem može da toleriše najviše $f = 1$ vizantijski čvor, a quorum iznosi 3 poruke.

2. Kratak opis IBFT algoritma

Algoritam se izvršava u rundama. U svakoj rundi postoji jedan lider koji predlaže vrednost. Ako je lider ispravan i mreža isporučuje poruke, čvorovi prolaze kroz tri osnovne faze: PRE-PREPARE, PREPARE i COMMIT. Kada čvor sakupi quorum COMMIT poruka za istu vrednost, donosi odluku.

U PRE-PREPARE fazi lider šalje predlog vrednosti. U PREPARE fazi čvorovi potvrđuju da su prihvatili predlog. Čvor postaje prepared tek kada sakupi quorum PREPARE poruka za istu instancu, istu rundu i istu vrednost. Tada pamti preparedRound i preparedValue. U COMMIT fazi čvorovi potvrđuju da su spremni da odluče, a quorum COMMIT poruka predstavlja Qcommit, odnosno dokaz odluke.

Ako se dogovor ne postigne u tekućoj rundi, tajmer ističe i pokreće se ROUND-CHANGE. Čvor tada prelazi u narednu rundu i šalje poruku koja sadrži preparedRound i preparedValue. Novi lider ne sme proizvoljno da predloži vrednost: ako u ROUND-CHANGE quorum-u postoji ranije pripremljena vrednost, bira se vrednost sa najvećim preparedRound. Ako takva vrednost ne postoji, novi lider može da predloži svoju input vrednost.

3. Osnovne informacije o implementaciji

Implementacija je organizovana kao Go projekat. Glavna simulacija se pokreće iz fajla `cmd/ibft-sim/main.go`, dok se sama logika IBFT algoritma nalazi u paketu `internal/ibft`. Na ovaj način je odvojena simulaciona logika, koja služi za unos parametara i pokretanje različitih scenarija, od logike samog konsenzus algoritma.

Osnovni element implementacije je čvor, predstavljen strukturom `Node`. Svaki čvor odgovara jednom procesu `pi` iz pseudokoda. U strukturi `Node` čuvaju se sve promenljive koje su potrebne za izvršavanje jedne instance IBFT konsenzusa: identifikator čvora, skup validatora, trenutna runda, identifikator instance konsenzusa, prepared stanje, primljene poruke, stanje odluke i stanje tajmera.

Najvažnije mapiranje između pojmova iz pseudokoda i implementacije dato je u tabeli:

Promenljiva/pojam iz algoritma	Realizacija u implementaciji
<code>p_i</code>	<code>Node.ID</code>
<code>lambda_i</code>	<code>Node.Lambda</code>
<code>r_i</code>	<code>Node.CurrentRound</code>
<code>pr_i</code>	<code>Node.PreparedRound</code>
<code>pv_i</code>	<code>Node.PreparedValue</code>
<code>inputValue_i</code>	<code>Node.InputValue</code>
<code>timer_i</code>	<code>TimerState + StartTimer/StopTimer/OnTimeout</code>
<code>leader(lambda, round)</code>	<code>Leader(lambda, round, validators)</code>
<code>Qcommit</code>	<code>DecisionCertificate</code>

Polje `Node.ID` predstavlja identifikator čvora. Polje `Node.Lambda` označava instancu konsenzusa, dok `Node.CurrentRound` označava trenutnu rundu u kojoj se čvor nalazi. Prepared stanje iz pseudokoda realizovano je pomoću polja `PreparedRound` i `PreparedValue`. Ako čvor još nije postao prepared, `PreparedRound` ima vrednost 0, a `PreparedValue` je prazan string. Na taj način se vrednost $\perp$ iz pseudokoda predstavlja u implementaciji.

Pored ovih osnovnih promenljivih, čvor čuva i mape primljenih poruka. Poruke tipa `PREPARE`, `COMMIT` i `ROUND-CHANGE` čuvaju se u odgovarajućim strukturama, kako bi čvor mogao da proveri da li je dostignut quorum. Na primer, čvor ne postaje prepared čim primi jednu `PREPARE` poruku, već tek kada broj primljenih validnih `PREPARE` poruka za istu instancu, rundu i vrednost dostigne quorum. Isto važi i za `COMMIT` poruke, gde se odluka donosi tek nakon prikupljanja quorum-a.

Procedura `Start(lambda, value)` predstavlja početak jedne instance konsenzusa. Ona postavlja identifikator instance `lambda`, trenutnu rundu na 1, resetuje prepared promenljive, briše prethodno zapamćene poruke i resetuje stanje odluke. Nakon toga se pokreće tajmer čvora. Ako je dati čvor lider prve runde, poziva se funkcija `Propose`, kojom se šalje `PRE-PREPARE`

poruka sa početnom vrednošću. Time je u implementaciji realizovan početni deo pseudokoda u kome lider prve runde predlaže vrednost ostalim procesima.

Izbor lidera je realizovan deterministički, pomoću funkcije `Leader(lambda, round, validators)`. Pošto u referentnom radu nije data konkretna implementacija funkcije za izbor lidera, u projektu je korišćeno round-robin pravilo. Za $\lambda = 1$ i validatore $[0, 1, 2, 3]$, lideri se smenjuju na sledeći način:

round 1 -> Node 0

round 2 -> Node 1

round 3 -> Node 2

round 4 -> Node 3

Ovakav izbor lidera je pogodan za simulaciju, jer svi ispravni čvorovi imaju isti skup validatora i istu vrednost trenutne runde. Zbog toga svaki čvor može nezavisno da izračuna ko je lider određene runde. Time se obezbeđuje determinističko ponašanje, što je u skladu sa zahtevom iz pseudokoda da funkcija `leader( $\lambda$ , r)` za iste ulaze daje isti rezultat svim ispravnim čvorovima.

Mreža je u implementaciji simulirana pomoću reda poruka. Funkcije `Send` i `Broadcast` ne šalju poruke kroz stvarnu mrežu, već ih ubacuju u red. Zatim funkcija `Network.Run` redom uzima poruke iz tog reda i isporučuje ih odgovarajućim čvorovima. Ovakav pristup je pogodan za edukativnu simulaciju, jer omogućava da se jasno vidi redosled događaja: ko šalje poruku, kome je poruka namenjena, kojoj instanci i rundi pripada i koju vrednost nosi.

Pored obične isporuke poruka, mreža podržava i simulaciju kašnjenja čvora. To je iskorišćeno u scenariju u kome jedan čvor privremeno ne prima poruke, a kasnije sustiže odluku pomoću `Qcommit` mehanizma. Na taj način se može demonstrirati situacija u kojoj neki ispravan čvor kasni, ali ipak može kasnije da proveriti dokaz odluke i dođe do iste vrednosti kao ostali ispravni čvorovi.

Važno je naglasiti da implementacija nije produkciona distribuirana mreža, već simulacija. Cilj nije bio da se napravi sistem koji radi preko stvarnih mrežnih konekcija, već da se jasno prikaže ponašanje IBFT algoritma, razmena poruka, formiranje quorum-a, promena runde i donošenje odluke. Zbog toga je ovakav model mreže dovoljan za prikaz i proveru osnovnih mehanizama algoritma.

Timer je takođe simuliran. U kodu postoje funkcije `StartTimer`, `StopTimer` i `OnTimeout`. Funkcija `StartTimer` postavlja tajmer u stanje `RUNNING`, funkcija `StopTimer` ga zaustavlja kada čvor donese odluku, dok `OnTimeout` simulira istek tajmera. Kada tajmer istekne, čvor prelazi u sledeću rundu i šalje `ROUND-CHANGE` poruku. Time je realizovan mehanizam promene runde iz pseudokoda. Vreme čekanja modelovano je funkcijom `TimeoutForRound(round)`, koja za veće runde daje veće vrednosti timeout-a, što prati ideju da se u delimično sinhronom sistemu vreme čekanja povećava kroz runde.

Ukratko, implementacija prati osnovne delove pseudokoda kroz sledeće funkcije: Start pokreće instancu konsenzusa, Propose šalje PRE-PREPARE, handlePrePrepare obrađuje predlog lidera, handlePrepare proverava quorum PREPARE poruka, handleCommit donosi odluku na osnovu quorum-a COMMIT poruka, dok handleRoundChange realizuje prelazak u novu rundu. Dodatno, funkcije HighestPrepared, JustifyRoundChange i JustifyPrePrepare koriste se za proveru opravdanosti poruka u slučaju promene runde.

Ovim je pseudokod iz rada preslikan u skup metoda nad čvorom i pomoćnih funkcija koje zajedno omogućavaju simulaciju normalnog toka algoritma, ponašanja u prisustvu vizantijskih čvorova, promene runde i sustizanja odluke pomoću Qcommit dokaza.

4. Najvažniji mehanizmi u kodu

Normalan tok: Propose šalje PRE-PREPARE, handlePrePrepare prihvata predlog lidera i šalje PREPARE, handlePrepare broji PREPARE poruke i pri quorum-u šalje COMMIT, a handleCommit pri quorum-u odlučuje vrednost.

Promena runde: OnTimeout povećava rundu i šalje ROUND-CHANGE. handleRoundChange skuplja ROUND-CHANGE poruke i proverava da li postoji opravdan Qrc za novu rundu.

f + 1 ROUND-CHANGE: Ako čvor primi $f + 1$ ROUND-CHANGE poruka za višu rundu, i sam prelazi u tu rundu i šalje ROUND-CHANGE. Za $n = 4$ važi $f + 1 = 2$.

HighestPrepared: Ako se u Qrc nalazi ranije prepared vrednost, novi lider bira vrednost sa najvećim PreparedRound. Ako takve vrednosti nema, koristi InputValue.

Justification: JustifyRoundChange proverava opravdanost ROUND-CHANGE quorum-a, a JustifyPrePrepare proverava da li je PRE-PREPARE poruka u višoj rundi opravdana Qrc dokazom.

Qcommit catch-up: Kada čvor odluči, čuva DecisionCertificate. Ako već odlučen čvor primi ROUND-CHANGE od čvora koji kasni, šalje mu Qcommit, pa zakasneli čvor može da sustigne odluku.

Validacija poruka: Svaka poruka ima simulirani potpis i proverava se funkcijom IsValidMessage. Beta(value) predstavlja pojednostavljenu proveru validnosti vrednosti.

5. Mogući scenariji simulacije

Program pri pokretanju traži broj čvorova, broj vizantijskih čvorova, režim vizantijskog ponašanja, način simulacije timeout-a i opciju za Qcommit catch-up scenario. Na taj način se može prikazati više različitih situacija.

Scenario	Primer unosa	Šta se proverava
Normalan rad	4, 0, timeout 1, catch-up 1	Svi čvorovi su ispravni i odluka se donosi u prvoj rundi.
BAD_VOTE	4, 1, mode 3, jedan vizantijski čvor	Vizantijski validator šalje PREPARE za pogrešnu vrednost.
EQUIVOCATE lider	4, 1, mode 2, Node 0	Vizantijski lider šalje različite PRE-PREPARE vrednosti i pokreće se ROUND-CHANGE.
f + 1 ROUND-CHANGE	EQUIVOCATE + timeout režim 2	Samo f+1 ispravnih čvorova dobija timeout, ostali prelaze kada prime f+1 ROUND-CHANGE poruka.
Previše vizantijskih	4, 3, mode 1 ili 2	Prekršen je uslov $n \geq 3f + 1$ i konsenzus ne može da se postigne.
Qcommit catch-up	4, 0, catch-up 2, npr. Node 3 kasni	Čvor koji kasni sustiže odluku preko Qcommit dokaza.

6. Rezultati značajnih scenarija

6.1 Normalan rad bez vizantijskih čvorova

U scenariju sa 4 čvora i 0 vizantijskih čvorova quorum iznosi 3. Lider prve runde je Node 0 i on predlaže vrednost block-1. Svi čvorovi prihvataju PRE-PREPARE, šalju PREPARE i pri 3/3 PREPARE poruka prelaze u PREPARED stanje. Zatim sakupljaju 3/3 COMMIT poruka i odlučuju istu vrednost.

Dokaz iz rezultata: Node 0/1/2/3 imaju state=DECIDED, preparedValue=block-1, preparedRound=1, qcommit=3, timer=STOPPED.

Ovo je očekivano ponašanje. Konsenzus se postiže u prvoj rundi, bez ROUND-CHANGE mehanizma. Safety provera je uspešna jer nijedna dva ispravna čvora nisu odlučila različite vrednosti.

6.2 Vizantijski čvor u BAD_VOTE režimu

U ovom scenariju postoji jedan vizantijski čvor, što je u granici tolerancije za $n = 4$. Vizantijski čvor šalje PREPARE za pogrešnu vrednost block-1-FAKE. Ispravni čvorovi su prethodno prihvatili vrednost block-1, pa ne broje PREPARE poruke za drugu vrednost. Zbog toga pogrešan glas ne ulazi u quorum.

Dokaz iz rezultata: Node 1, Node 2 i Node 3 sakupljaju 3/3 PREPARE i 3/3 COMMIT za block-1, a zatim odlučuju block-1 uz $Qcommit=3$.

Rezultat je ispravan: iako jedan čvor šalje pogrešan glas, tri ispravna čvora formiraju quorum za istu vrednost. Konsenzus je postignut, a safety ostaje očuvana.

6.3 Vizantijski lider u EQUIVOCATE režimu i f+1 ROUND-CHANGE

U ovom scenariju Node 0 je vizantijski lider i različitim čvorovima šalje različite PRE-PREPARE vrednosti: nekima block-1-A, a nekima block-1-B. Zbog toga ispravni čvorovi ne uspevaju da sakupe quorum PREPARE poruka za jednu istu vrednost u prvoj rundi.

Dokaz iz rezultata: Node 1 ima 2/3 PREPARE za block-1-B, Node 2 ima 1/3 za block-1-A, a quorum 3/3 nije postignut u round 1.

Pošto konsenzus nije postignut, simulira se timeout. U posebnom f+1 scenariju timeout ističe samo kod dva ispravna čvora. Za $n = 4$ važi $f = 1$, pa je $f + 1 = 2$. Kada Node 3 primi dve ROUND-CHANGE poruke za višu rundu, i sam prelazi u round 2 i šalje ROUND-CHANGE.

Dokaz iz rezultata: Node 3 je primio f+1 ROUND-CHANGE poruka za visu rundu 2 i prelazi u nju.

Nakon toga Node 1, kao lider druge runde, dobija opravdan Qrc. Pošto u ROUND-CHANGE porukama nema prepared vrednosti ($pr=0$, $pv=\perp$), Node 1 koristi inputValue block-1. U drugoj rundi ispravni čvorovi prolaze kroz PREPARE i COMMIT faze i odlučuju block-1. Ovo pokazuje da round-change i f+1 pravilo omogućavaju oporavak od neuspešne runde sa vizantijskim liderom.

6.4 Previše vizantijskih čvorova

U scenariju sa 4 čvora i 3 vizantijska čvora prekršen je osnovni uslov IBFT algoritma. Za $n = 4$ sistem može da toleriše najviše jednog vizantijskog čvora, a quorum je 3. Kada postoje tri vizantijska čvora, ostaje samo jedan ispravan čvor, pa nije moguće sakupiti quorum PREPARE, COMMIT ili ROUND-CHANGE poruka.

Dokaz iz rezultata: Node 3 ima samo 1/3 PREPARE i kasnije 1/3 ROUND-CHANGE poruka, pa odluka nije doneta.

Rezultat je očekivan: konsenzus nije postignut. Ovo nije greška implementacije, već posledica nevažeće konfiguracije. Safety je i dalje očuvana, jer nijedan ispravan čvor nije doneo pogrešnu ili različitu odluku; problem je u liveness/termination osobini.

6.5 SILENT lider i promena runde

U scenariju sa jednim vizantijskim čvorom u SILENT režimu lider prve runde ne predlaže vrednost. Pošto nema PRE-PREPARE poruke, čvorovi ne mogu da započnu normalan tok algoritma u round 1. Nakon timeout-a ispravni čvorovi šalju ROUND-CHANGE i prelaze u round 2.

Dokaz iz rezultata: Node 0 je vizantijski lider u SILENT režimu i ne predlaže vrednost; zatim Node 1, Node 2 i Node 3 šalju ROUND-CHANGE za round 2.

Node 1 postaje lider druge runde, dobija opravdan Qrc i predlaže block-1. Ispravni čvorovi zatim sakupljaju quorum PREPARE i COMMIT poruka i odlučuju block-1. Rezultat je ispravan jer IBFT može da toleriše jednog neispravnog lidera i da se oporavi promenom runde.

6.6 Qcommit catch-up scenario

Poseban scenario pokazuje ponašanje čvora koji privremeno kasni. U toj simulaciji jedan ispravan čvor ne prima početne poruke, dok ostali čvorovi uspevaju da odluče. Kada se zakasneli čvor vrati u mrežu i pošalje ROUND-CHANGE, već odlučeni čvorovi mu šalju Qcommit, odnosno decision certificate. Na osnovu tog dokaza zakasneli čvor može da sustigne odluku bez ponavljanja celog protokola.

Dokaz iz rezultata: Node 0/1/2 šalju Qcommit čvoru koji kasni, a zatim se ispisuje da je zakasneli čvor sustigao odluku preko Qcommit: block-1.

Ovaj scenario potvrđuje da se u implementaciji ne pamti samo vrednost odluke, već i skup COMMIT poruka koje dokazuju odluku. To je važno za čvorove koji kasne ili su privremeno bili izolovani iz mreže.

7. Ograničenja simulacije

- Mreža je simulirana redom poruka, a ne stvarnim mrežnim konekcijama.
- Timeout se ne čeka u realnom vremenu, već se eksplicitno simulira iz programa.
- Digitalni potpis je pojednostavljen i služi samo da prikaže ideju autentifikacije poruka.
- Beta(value) je jednostavna provera, a u pravom blockchain sistemu bi predstavljala validaciju bloka i transakcija.
- Vizantijska ponašanja su unapred definisana kroz režime SILENT, EQUIVOCATE i BAD_VOTE.
- maxRounds je dodat samo da se simulacija zaustavi kada konfiguracija ne može da postigne konsenzus.

8. Zaključak

Implementacija prikazuje ključne delove IBFT algoritma: normalan tok kroz PRE-PREPARE, PREPARE i COMMIT faze, round-change mehanizam, $f+1$ ROUND-CHANGE pravilo, izbor vrednosti preko HighestPrepared, justification provere, Qcommit dokaz odluke i catch-up mehanizam za čvor koji kasni.

Rezultati simulacija pokazuju očekivano ponašanje: u ispravnoj konfiguraciji sistem postiže konsenzus, u prisustvu jednog vizantijskog čvora safety ostaje očuvana, a kada je broj vizantijskih čvorova veći od dozvoljenog konsenzus se ne postiže. Na taj način projekat demonstrira osnovne osobine IBFT algoritma i vezu između teorijskog pseudokoda i konkretne Go implementacije.

Implementacija predstavlja edukativnu simulaciju IBFT algoritma. Mreža je modelovana redom poruka, timeout se simulira eksplicitnim pozivima, a digitalni potpisi su pojednostavljeni. Cilj implementacije nije produkcionu blockchain sistem, već prikaz toka IBFT algoritma i njegovih ključnih mehanizama: normalnog toka, promene runde, opravdanja poruka i Qcommit mehanizma.